Prefix-aware routing

Optimize LLM inference with cache locality using prefix-aware request routing.

Warning

This API is in alpha and may change before becoming stable.

LLM inference can benefit significantly from cache locality optimization. When one replica processes multiple prompts that share a prefix, the engine can reuse previously computed KV-cache entries, reducing computation overhead and improving response times. This technique is known as Automatic Prefix Caching (APC) in vLLM.

The `PrefixCacheAffinityRouter` routes requests with similar prefixes to the same replicas, maximizing KV cache hit rates.

When to use prefix-aware routing

Use prefix-aware routing when:

- Your workload has many requests with shared prefixes (for example, same system prompts or few-shot examples)
- You're using vLLM with Automatic Prefix Caching enabled
- Cache hit rate is more important than perfect load balance in balanced scenarios

How it works

The `PrefixCacheAffinityRouter` implements a multi-tier routing strategy that balances cache locality with load distribution:

1. Load balance check

First, it evaluates whether the current load is balanced across replicas by comparing queue lengths. If the difference between the highest and lowest queue lengths is below the `imbalanced_threshold`, it proceeds with prefix cache-aware routing.

2. Prefix matching strategy

When load is balanced, the router uses a prefix tree to find replicas that have previously processed similar input text:

- **High match rate (≥10%)**: Routes to replicas with the highest prefix match rate for better cache hit rates
- **Low match rate (<10%)**: Falls back to replicas with the lowest prefix cache utilization to increase utilization
- **No prefix data**: Uses the default Power of Two Choices selection

3. Imbalanced load fallback

When load is imbalanced (queue length difference exceeds threshold), the router prioritizes load balancing over cache locality and falls back to the standard Power of Two Choices algorithm.

The router maintains a distributed prefix tree actor that:

- Tracks input text prefixes processed by each replica
- Supports automatic eviction of old entries to manage memory usage
- Persists across router instances using Ray's detached actor pattern

Deploy with prefix-aware routing

The following example shows how to deploy an LLM with prefix-aware routing:

```
from ray import serve
from ray.serve.llm import LLMConfig, build_openai_app
from ray.serve.llm.request_router import PrefixCacheAffinityRouter

llm_config = LLMConfig(
    model_loading_config={
        "model_id": "qwen-0.5b",
        "model_source": "Qwen/Qwen2.5-0.5B-Instruct",
    },
    deployment_config={
        "autoscaling_config": {
            "min_replicas": 4,
            "max_replicas": 4,
        },
        "request_router_config": {
            "request_router_class": PrefixCacheAffinityRouter,
            "request_router_kwargs": {
                "imbalanced_threshold": 5,  # More aggressive load balancing
                "match_rate_threshold": 0.15,  # Require 15% match rate
                "do_eviction": True,  # Enable memory management
                "eviction_threshold_chars": 500_000,
                "eviction_target_chars": 400_000,
                "eviction_interval_secs": 30,
            },
        },
    },
    runtime_env={"env_vars": {"VLLM_USE_V1": "1"}},
)

app = build_openai_app({"llm_configs": [llm_config]})
serve.run(app, blocking=True)
```

Configuration parameters

The `PrefixCacheAffinityRouter` provides several configuration parameters to tune its behavior:

Core routing parameters

- `imbalanced_threshold` (default: infinity): Queue length difference threshold for considering load balanced. Lower values prioritize load balancing over cache locality.
- `match_rate_threshold` (default: 0.1): Minimum prefix match rate (0.0-1.0) required to use prefix cache-aware routing. Higher values require stronger prefix matches before routing for cache locality.

Memory management parameters

- `do_eviction` (default: False): Enable automatic eviction of old prefix tree entries to approximate the LLM engine's eviction policy.
- `eviction_threshold_chars` (default: 400,000): Maximum number of characters in the prefix tree before the LLM engine triggers an eviction.
- `eviction_target_chars` (default: 360,000): Target number of characters to reduce the prefix tree to during evicti
- `eviction_interval_secs` (default: 10): Interval in seconds between eviction checks when eviction is enabled.

Skip to main content

Best practices

- **Enable vLLM APC**: Make sure to set `enable_prefix_caching=True` in your `engine_kwargs` for the router to have any effect
- **Tune thresholds**: Adjust `imbalanced_threshold` and `match_rate_threshold` based on your workload characteristics
- **Monitor cache hit rates**: Track vLLM's cache hit metrics to verify the router is improving performance
- **Start conservative**: Begin with default settings and tune incrementally based on observed behavior

See also

- Architecture: Request routing
- vLLM Automatic Prefix Caching

Previous
KV cache offloading

Next
Multi-LoRA deployment